

决故障并恢复服务，同时更新有关故障信息。

1. 硬件设备故障的恢复

主机故障需要启用系统备份进行恢复，对于比较重要的系统，一般采用冗余的结构，如主备或双活，抑或以资源池方式提供保护。当系统出现故障时，就用一个备用的模块或另外处于同等功能的主机，或资源池里的其他主机来顶替它并重新运行。

当遇到线路故障或者网络连接问题时，需要利用备用电路或者改变通信路径等恢复方法。针对系统其他相关设备的故障，应分析、查找设备有关技术与非技术上的故障原因，如有需要应与供应商取得联系，进行设备维修、调换、更新后，使设备运行正常。

2. 数据库故障的恢复

数据库故障主要分为事务故障、系统故障和介质故障，不同故障的恢复方法也不同。当系统运行过程中发生故障，利用数据库后备副本和日志文件就可以将数据库恢复到故障前的某一一致性状态。

3. 应用软件故障的恢复

信息系统中的应用软件由于设计不当，或者程序代码有误，都会造成系统故障。对于软件类故障的处理方法，通常是通过软件调试，找出错误的代码，然后进行修改和测试。为了提高系统的可靠性，也可以采取软件容错技术。

15.3.4 故障收尾

解决故障和恢复服务后，就到了故障收尾阶段。该阶段主要是基于上一阶段更新后的故障记录和已解决的故障，与用户一起确认故障是否被成功解决，并更新故障信息和故障记录。

15.4 软件系统维护

系统维护是系统运行过程中的重要一环，是保证系统正常运行，或为了改正错误或满足新需求而修改系统的活动，包括软件维护（程序维护）、数据维护、代码维护、设备维护，以及机构和人员的变动等。在整个信息系统的维护过程中，软件维护是最重要的工作，也是最难的工作。因此，本节只讨论软件维护的问题。

15.4.1 软件维护概述

在系统交付使用后，改变系统的任何工作都可以被称为维护。与硬件不同，软件系统在构建时就包含了变化，软件并不会老化或需要周期性的维护。软件维护是指在软件交付使用之后，直至软件被淘汰的整个时期内，为了改正错误或满足新的需求而修改软件的活动。软件的维护活动基于“软件是可维护的”这一基本前提。

1. 软件维护性

根据国家标准《系统与软件工程 系统与软件质量要求和评价（SQuaRE）第10部分：系统

与软件质量模型》（GB/T 25000.10—2016），软件维护性是指产品或系统能够被预期的维护人员修改的有效性和效率的程度。修改包括纠正、改进或软件对环境、需求和功能规格说明变化的适应。修改还包括那些由专业支持人员实施的，以及那些由业务或操作人员、最终用户实施的。维护性包括安装更新和安装升级。维护性可以被解释为便于维护活动的一种产品或系统固有能力，或者为了产品或系统维护的目标维护人员所经历的使用质量。

GB/T 25000.10—2016 标准还规定了维护性的6个子特性：

（1）模块化。指由多个独立组件组成的系统或计算机软件，其中一个或多个组件的变更对其他组件产生的影响最小的程度。

（2）可重用性。软件能够被用于多个系统或其他软件构建的程度。

（3）易分析性。可以评估预期变更（变更产品或系统的一个或多个部分）对产品或系统的影响、诊断产品的缺陷或失效原因、识别待修改部分的有效性和效率的程度。

（4）易修改性。产品或系统可以被有效地、有效率地修改，且不会引入缺陷或降低现有产品质量的程度。（注：修改包括编码、设计、文档和验证的变更。模块化和易分析性会影响易修改性。）

（5）易测试性。能够为产品、系统或组件建立测试准则，并通过测试执行来确定测试准则是否被满足的有效性和效率的程度。

（6）维护性的依从性。产品或系统遵循与维护性相关的标准、约定或法规以及类似规定的程度。

2. 可维护性度量

人们一直期望对软件的可维护性做出定量度量，但要做到这一点并不容易。国家标准《系统与软件工程 系统与软件质量要求与评价（SQuaRE）第23部分：系统与软件产品质量测量》（GB/T 25000.23—2019）具体规定了如何从软件的外部 and 内部两个方面来度量可维护性，但是，这些规定比较烦琐，实用性比较差。因此，下面从实用角度来介绍可维护性的度量方法。

在软件外部，可以用 MTTR 来度量软件的可维护性，它指出处理一个有错误的软件需要花费的平均时间。为此，需要详细记录分析问题需要的时间、确定改动方案的时间、执行改动花费的时间、测试改动花费的时间，以及其他管理花费的时间。

在软件内部，可以通过度量软件的复杂性来间接度量可维护性。与软件复杂性相关的因素有环路数、软件规模等。

（1）环路数。环路数是 McCabe 在 1976 年提出的，因此也称为 McCabe 环路，它基于一个程序模块的程序图中环路的个数来度量软件复杂性。环路数可以反映源代码结构的复杂度，通过观察环路数的增长幅度，可以比较多种维护方案的优劣，选择对环路数影响最小的方案作为最优方案。

（2）软件规模。通常可以认为软件包含的构件越多，软件就越复杂，可维护性就越差。

（3）其他因素。其他因素包括嵌套深度、系统用户数等。

对于软件内部的可维护性，迄今还没有一个典型的、全面的、通用的模型，需要结合